LCD Display

Department of Electrical and Electronic Engineering,

Universiti Teknologi PETRONAs



- Adapted from
- ***Richard Kuo***
- *Assistant Professor*
- *Department of Electrical Engineering*
- *National Taiwan Ocean University*

Outline

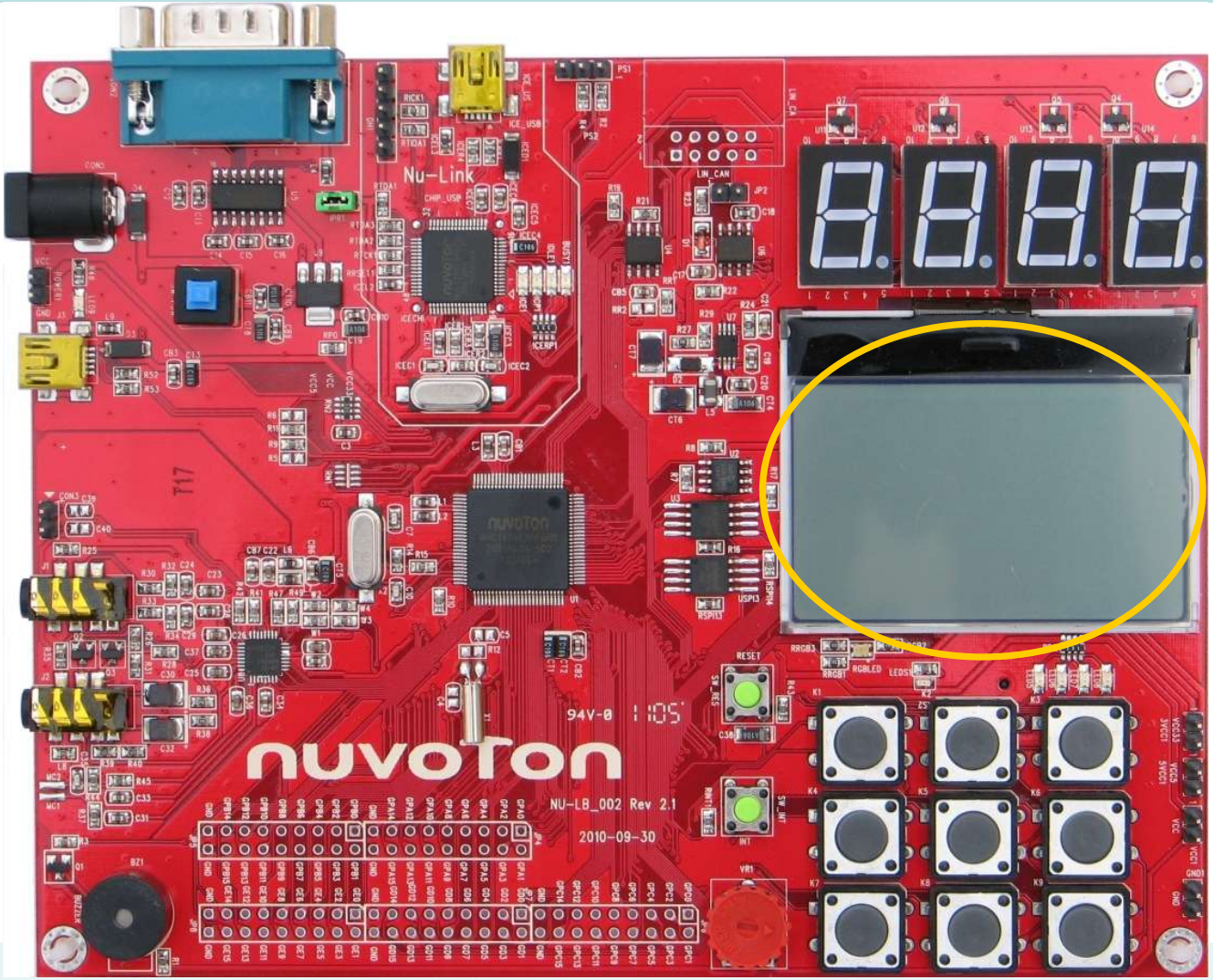
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- **LCD Display Introduction** 4.NuMicro_LCD.ppt
- **Exercise: LCD Text mode (Smpl_LCD_Text)**
- **Exercise: LCD Text with keypad (Smpl_LCD_Keypad)**
- **Exercise: LCD Text with Timer (Smpl_LCD_Timer)**
- **Exercise: LCD display Bitmap of Logo (Smpl_LCD_Bmp)**
- **Exercise: LCD display Bitmap of Arrows (Smpl_LCD_Bmp_Arrows)**
- **Exercise: LCD display Bitmap of Cartoon (Smpl_LCD_Bmp_Cartoon)**
- **Exercise: LCD draw bitmap of object/icon (Smpl_LCD_BmpDraw)**
- **Exercise: LCD move bitmap of object/icon (Smpl_LCD_BmpMove)**
- **Exercise: LCD draw 2D graphics (Smpl_LCD_Graph_2D)**
- **Exercise: LCD draw sin/cosine waveform (Smpl_LCD_Graph_SinCos)**
- **Project: LCD Game / Animation**

Note: Smpl_TFT_* is for color TFT (320x240)

Nu-LB-NUC140 Learning Board

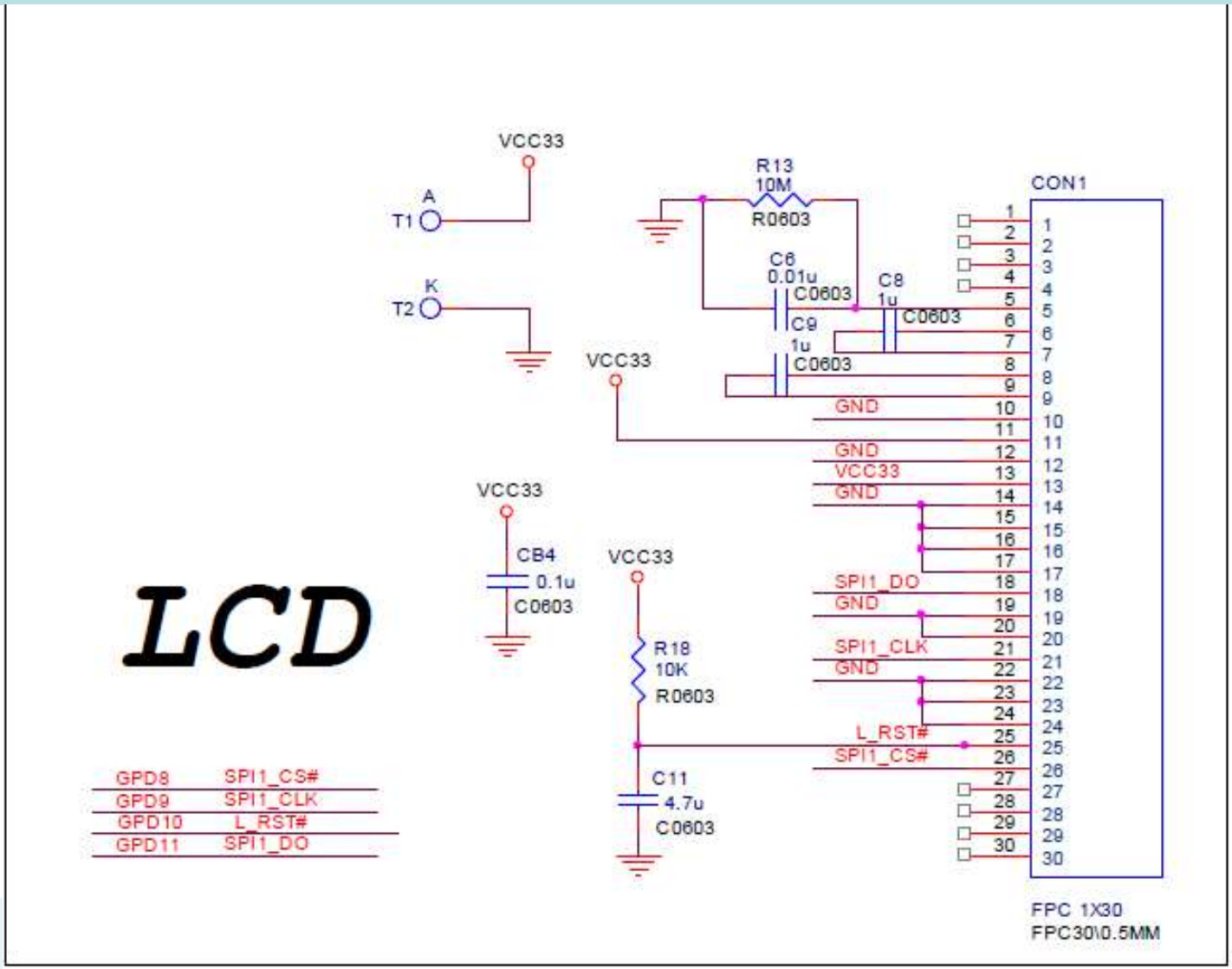
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



LCD
128x64 pixels
SPI interface

LCD schematic

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



SPI Pins used on Nu-LB-NUC140 board

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

SPI signals	SPI0	SPI1	SPI2	SPI3
CS	GPC0	GPC8	GPD0	GPD8
CLK	GPC1	GPC9	GPD1	GPD9
DI0 (MISO)	GPC2	GPC10	GPD2	GPD10
DO0 (MOSI)	GPC3	GPC11	GPD3	GPD11
DI1	GPC4	GPC12	GPD4	GPD12
DO1	GPC5	GPC13	GPD5	GPD13
Nu-LB-NUC140 On-Board	**I2S Codec** (WAU8822)	**SD/MMC**	**EEPROM** (W25Q16CV)	**LCD** 128x64

LCD Module Interface Signals

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

Interface			
Pin no	Symbol	Pin no	Symbol
1	NC	16	D5
2	NC	17	D4
3	NC	18	D3/SDA
4	NC	19	D2
5	VLCD	20	D1
6	VB0+	21	D0/SCK
7	VB0-	22	WR1
8	VB1-	23	WR0
9	VB1+	24	CD
10	VSS	25	RST
11	VDD	26	/CS0
12	BM1	27	NC
13	BM0	28	NC
14	D7	29	NC
15	D6	30	NC

Pin No	Symbol	I/O	Function
1~4	NC	-	No Connection.
5	V_{LCD}	PWR	Main LCD power supply.
6	V_{B0+}	PWR	LCD Bias Voltages. These are the voltage sources to provide SEG driving currents. These voltages are generated internally. Connect capacitors of C_{BX} value between V_{BX+} and V_{BX-}.
7	V_{B0-}	PWR	
8	V_{B1-}	PWR	
9	V_{B1+}	PWR	
10	VSS	PWR	Power Ground.
11	VDD	PWR	Power supply terminal VCC.
12	BM1	I	Bus mode: "HL": 8080 "HH": 6800
13	BM0	I	BM[1:0] "LH": S9 "LL": S8
14	DB7	I/O	Bi-directional bus for both serial and parallel host interfaces. In serial modes, connect DB0 to SCK, DB3 to SDA.
15	DB6	I/O	
16	DB5	I/O	
17	DB4	I/O	
18	DB3/SDA	I/O	
19	DB2	I/O	
20	DB1	I/O	
21	DB0/SCK	I/O	
22	WR1	I	WR [1:0] controls the read/write operation of the host interface. See Host Interface section for details. The meaning of WR [1:0] depends on whether the interface is in the 6800 mode, or the 8080 mode. In serial modes, these two pins are not used and can be connected to Vss.
23	WR0	I	
24	CD	I	Select the incoming command if it is a control instruction or for display data. CD pin is not used in S9 mode, connect it to Vdd or Vss. " L": control instruction "H": display data
25	RST	I	When RST="L", all control registers are re-initialized by their default states.
26	/CS0	I	Chip Select or Chip Address. In parallel mode and S8 mode, chip is selected when /CS0="L". When the chip is not selected, DB[7:0] will be high impedance.
27~30	NC	-	No Connection.

	BM=1x (Parallel)	BM=0x (Serial)
D0	D0	SCK
D1	D1	
D2	D2	
D3	D3	SDA
D4	D4	
D5	D5	
D6	D6	-
D7	D7	-

LCD module

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

Bus Type		8080	6800	SPI (S8)	SPI (S9)
Control & Data Pins	BM[1:0]	10b	11b	00b	01b
	CS[1:0]	Chip Select			
	CD	Control/Data			–
	WR0	$\overline{WR}$	R/$\overline{W}$	–	
	WR1	$\overline{RD}$	EN	–	
	Access	Read/Write		Write Only	
	D[7:0]	8-bit bus (Tri-state)		D0=SCK, D3=SDA	

- http://www.computex.biz/PhotoPool2/201103/20113111045404.pdf
- http://www.datamate-j.com/wp-content/uploads/2013/06/GFG128064A-FPFAB001SPEC.pdf

LCD Driver Header File

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Include\LCD.h

```
#define LCD_Xmax 128
#define LCD_Ymax 64
#define FG_COLOR 0xFFFF
#define BG_COLOR 0x0000

extern void init_LCD(void);
extern void clear_LCD(void);
extern void printC(int16_t x, int16_t y, unsigned char ascii_code);
extern void printC_5x7(int16_t x, int16_t y, unsigned char ascii_code);
extern void print_Line(int8_t line, char text[]);
extern void printS(int16_t x, int16_t y, char text[]);
extern void printS_5x7(int16_t x, int16_t y, char text[]);
```

LCD Driver Header File

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

draw bitmap

```
extern void draw_Bmp8x8(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp16x8(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp16x16(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp16x32(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp16x48(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp16x64(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp32x16(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp32x32(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp32x48(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp32x64(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);
extern void draw_Bmp64x64(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor, unsigned char bitmap[]);

extern void draw_Pixel(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor);

extern void draw_LCD(unsigned char *buffer);
```

draw single pixel

draw entire LCD (128x64 pixels)

Note: LCD function calls are written to be compatible to TFT !

LCD Driver Source Code

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\LCD.c

```
#include "stdio.h"
#include "string.h"
#include "NUC1xx.h"
#include "SYS.h"
#include "SPI.h"
#include "GPIO.h"
#include "LCD.h"
#include "Font5x7.h"
#include "Font8x16.h“

extern SPI_T * SPI_PORT[4]={SPI0, SPI1, SPI2, SPI3};

char DisplayBuffer[128*8];
```

5x7 font table for ascii code

8x16 font table for ascii code

buffer for drawing pixel on LCD
due to it is byte operation (write 8-pixel)
so need to keep track of which bits were written

LCD Driver Source Code

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\LCD.c

```
void init_SPI3(void)
{
        DrvGPIO_InitFunction(E_FUNC_SPI3);
        /* Configure SPI3 as a master, Type1 waveform, 32-bit transaction */
        DrvSPI_Open(eDRVSPI_PORT3, eDRVSPI_MASTER, eDRVSPI_TYPE1, 9);
        /* MSB first */
        DrvSPI_SetEndian(eDRVSPI_PORT3, eDRVSPI_MSB_FIRST);
        /* Disable the automatic slave select function of SS0. */
        DrvSPI_DisableAutoSS(eDRVSPI_PORT3);
        /* Set the active level of slave select. */
        DrvSPI_SetSlaveSelectActiveLevel(eDRVSPI_PORT3,
        eDRVSPI_ACTIVE_LOW_FALLING);
        /* SPI clock rate 1MHz */
        DrvSPI_SetClockFreq(eDRVSPI_PORT3, 25000000, 0);
}
```

Initilize SPI3 for interfacing to LCD

Type1 waveform
Clock rising edge at the middle of read/write data

Set SPI3 at 25MHz

LCD Driver Source Code

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\LCD.c

```
void lcdWriteCommand(unsigned char temp)
{
        // Write Data
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=1;             //chip select
        SPI_PORT[eDRVSPI_PORT3]->TX[0]=temp;            //write command
        SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY = 1;
        while ( SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY == 1 ); //check data out?
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=0;
        while(DrvSPI_IsBusy(eDRVSPI_PORT3) != 0);
}
```

LCD Driver Source Code

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\LCD.c

```
// Wrtie data to LCD
void lcdWriteData(unsigned char temp)
{
        // Write Data
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=1;             //chip select
        SPI_PORT[eDRVSPI_PORT3]->TX[0] =0x100 | temp;    //write data
        SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY = 1;
        while ( SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY == 1 ); //check data out?
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=0;
}
```

LCD Driver Source Code

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\LCD.c

```
// Set Address to LCD
void lcdSetAddr(unsigned char PA, unsigned char CA)
{
        // Set PA
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=1;
        SPI_PORT[eDRVSPI_PORT3]->TX[0] = 0xB0 | PA;
        SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY = 1;
        while ( SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY == 1 );
        // Set CA MSB
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=0;
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=1;
        SPI_PORT[eDRVSPI_PORT3]->TX[0] =0x10 |(CA>>4)&0xF;
        SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY = 1;
        while ( SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY == 1 );
        // Set CA LSB
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=0;

        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=1;
        SPI_PORT[eDRVSPI_PORT3]->TX[0] =0x00 | (CA & 0xF);
        SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY = 1;
         while ( SPI_PORT[eDRVSPI_PORT3]->CNTRL.GO_BUSY == 1 );
        SPI_PORT[eDRVSPI_PORT3]->SSR.SSR=0;
}
```

ASCII Code for Charactors

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

表2-14 七位元的ASCII碼

列	行 0	1	2	3	4	5	6	7
	BITS 4321 765	000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	、	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	”	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
10	1010	LT	SUB	*	:	J	Z	j	z
11	1011	VT	ESC	+	;	K	〔	k	{
12	1100	FF	FS	,	<	L	\	l	|
13	1101	CR	GS	-	=	M	〕	m	}
14	1110	SO	RS	.	>	N	(↑)	n	~
15	1111	SI	US	/	?	O	←(←)	o	DEL

Smpl_LCD_Text

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
int main(void)
{
    UNLOCKREG();
    DrvSYS_Open(50000000); // set to 50MHz
    LOCKREG();

    init_LCD();
    clear_LCD();

    print_Line(0, "Smpl_LCD_Text   ");
    print_Line(1, "Nu-LB-NUC140    ");
    print_Line(2, "Test LCD Display");
    print_Line(3, "Nuvoton NuMicro ");
}
```



Smpl_LCD_Keypad

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



Smpl_LCD_Bmp

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



Display bitmap

Step 1. use Paint to draw a **128x64 bitmap**
Step 2. use **bmp2asm** to convert bitmap into hex
Step 3. **cut and paste hex of bitmap to display buffer** in Smpl_LCD_Bmp.c

Note: In order to draw bitmap, draw_LCD() should be added into LCD_Driver.c

draw_LCD()

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
// display buffer = 128x8 bytes (X=128, Y= 8 *8-bit)
void draw_LCD(unsigned char *buffer)
{
  uint8_t x,y;
            for (x=0; x<LCD_Xmax; x++) {
            for (y=0; y<(LCD_Ymax/8); y++) {
                                    lcdSetAddr(y,(LCD_Xmax+1-x));
                                    lcdWriteData(buffer[x+y*LCD_Xmax]);
                                  }
                          }
}
```

Set X,Y to LCD register

DisplayBuffer Data write to LCD

main()


```
int32_t main (void)
{
        UNLOCKREG();
        DrvSYS_SetOscCtrl(E_SYS_XTL12M,1);
        DrvSYS_Delay(20000);
        while(!SYSCLK->CLKSTATUS.XTL12M_STB);
         DrvSYS_SelectHCLKSource(0);
        LOCKREG();

        init_LCD();
        clear_LCD();

        while(1) {
          draw_LCD(BMP_Logo_Nuvoton);
          DrvSYS_Delay(500000);
          draw_LCD(BMP_Logo_University);
          DrvSYS_Delay(500000);
        }
}
```

setting CPU clock

Initialize & clear LCD

write nuvoton logo bitmap to LCD

write university logo bitmap to LCD

Smp_LCD_Graph_2D

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



Note: In order to draw graphics, 2D_Graphic_Driver.c is needed !

draw_Pixel(x,y, fgColor, bgColor)

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

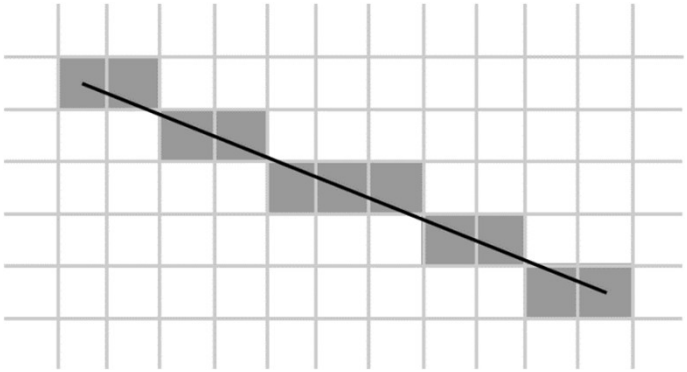
```
// draw pixel into Display Buffer
void draw_Pixel(int16_t x, int16_t y, uint16_t fgColor, uint16_t bgColor)
{
    if (fgColor!=0)
        DisplayBuffer[x+y/8*LCD_Xmax] |= (0x01<<(y%8));
    else
        DisplayBuffer[x+y/8*LCD_Xmax] &= (0xFE<<(y%8));

    lcdSetAddr(y/8,(LCD_Xmax+1-x));
    lcdWriteData(DisplayBuffer[x+y/8*LCD_Xmax]);
}
```

Bresenham Line-Drawing Algorithm

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- The Bresenham Line-Drawing Algorithm
- 布雷森漢姆直線演算法
- Line equation : $y - y_0 = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$
  - so, $y = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0) + y_0$



- Consider drawing a line on a raster grid where we restrict the allowable slopes of the line to the range . If we further restrict the line-drawing routine so that it always increments *x* as it plots, it becomes clear that, having plotted a point at *(x,y)*, the routine has a severely limited range of options as to where it may put the *next* point on the line:
- It may plot the point *(x+1,y)*, or:
- It may plot the point *(x+1,y+1)*.

Use line slope to determine x,y increment to draw the pixel

2DGraphics.h (Driver Header File)

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Include\2DGraphics.h

```
extern void LineBresenham(int x1, int y1, int x2, int y2, uint16_t fg_color, uint16_t bg_color);
extern void LineOptimized(int x1, int y1, int x2, int y2, uint16_t fg_color, uint16_t bg_color);

extern void CircleBresenham(int xc, int yc, int r, uint16_t fg_color, uint16_t bg_color);
extern void CircleMidpoint(int xc, int yc, int r, uint16_t fg_color, uint16_t bg_color);
extern void CircleOptimized(int xc, int yc, int r, uint16_t fg_color, uint16_t bg_color);

extern void RectangleDraw(int x0, int y0, int x1, int y1, uint16_t fg_color, uint16_t bg_color);
extern void RectangleFill(int x0, int y0, int x1, int y1, uint16_t fg_color, uint16_t bg_color);

extern void Triangle (int x0, int y0, int x1, int y1, int x2, int y2, uint16_t fg_color, uint16_t bg_color);
```

2DGraphics.c (Driver Source Code)

LineBresenham(int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t fgColor, uint16_t bgColor)

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\2DGraphics.c

```
void LineBresenham(int16_t x1, int16_t y1, int16_t x2, int16_t
      y2, uint16_t fgColor, uint16_t bgColor)
{
  int16_t dy = y2 - y1;
  int16_t dx = x2 - x1;
  int16_t stepx, stepy;

  if (dy < 0) { dy = -dy;  stepy = -1; } else { stepy = 1; }
  if (dx < 0) { dx = -dx;  stepx = -1; } else { stepx = 1; }
  dy <<= 1;       // dy is now 2*dy
  dx <<= 1;       // dx is now 2*dx

  draw_Pixel(x1,y1, fgColor, bgColor);
  if (dx > dy)
  {
    int fraction = dy - (dx >> 1);  // same as 2*dy - dx
    while (x1 != x2)
    {
      if (fraction >= 0)
      {
        y1 += stepy;
        fraction -= dx;         // same as fraction -= 2*dx
      }
      x1 += stepx;
      fraction += dy;           // same as fraction -= 2*dy
      draw_Pixel(x1, y1, fgColor, bgColor);
    }
  } else {
    int fraction = dx - (dy >> 1);
    while (y1 != y2) {
      if (fraction >= 0) {
        x1 += stepx;
        fraction -= dy;
      }
      y1 += stepy;
      fraction += dx;
      draw_Pixel(x1, y1, fgColor, bgColor);
    }
  }
}
```

based on Bresenham algorithm

CircleBresenham(int16_t xc, int16_t yc, int16_t r, uint16_t fgColor, uint16_t bgColor)

Department of Electrical and Electronic Engineering, Universiti Teknologi PETRONAS

```
void CircleBresenham(int16_t xc, int16_t yc, int16_t r, uint16_t fgColor, uint16_t bgColor)
{
   int16_t x = 0;
   int16_t y = r;
   int16_t p = 3 - 2 * r;
   if (!r) return;
   while (y >= x) // only formulate 1/8 of circle
   {
      draw_Pixel(xc-x, yc-y, fgColor, bgColor);//upper left left
      draw_Pixel(xc-y, yc-x, fgColor, bgColor);//upper upper left
      draw_Pixel(xc+y, yc-x, fgColor, bgColor);//upper upper right
      draw_Pixel(xc+x, yc-y, fgColor, bgColor);//upper right right
      draw_Pixel(xc-x, yc+y, fgColor, bgColor);//lower left left
      draw_Pixel(xc-y, yc+x, fgColor, bgColor);//lower lower left
      draw_Pixel(xc+y, yc+x, fgColor, bgColor);//lower lower right
      draw_Pixel(xc+x, yc+y, fgColor, bgColor);//lower right right
      if (p < 0) p += 4*(x++) + 6;
      else p += 4*((x++) - y--) + 10;
    }
}
```

http://willperone.net/Code/codecircle.php

RectangleDraw & RectanlgeFill

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

D:\Nuvoton\NUC140BSP_EDU\Library\NUC1xx-LB_002\Source\2DGraphics.c

```
void RectangleDraw(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t fgColor, uint16_t bgColor)
{
        int16_t x,y, tmp;
        if (x0>x1) { tmp = x1; x1 = x0; x0 = tmp; }
        if (y0>y1) { tmp = y1; y1 = y0; y0 = tmp; }
        for (x=x0; x<=x1; x++) draw_Pixel(x,y0,fgColor, bgColor);
        for (y=y0; y<=y1; y++) draw_Pixel(x0,y,fgColor, bgColor);
        for (x=x0; x<=x1; x++) draw_Pixel(x,y1,fgColor, bgColor);
        for (y=y0; y<=y1; y++) draw_Pixel(x1,y,fgColor, bgColor);
}

void RectangleFill(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t fgColor, uint16_t bgColor)
{
        int16_t x,y, tmp;
        if (x0>x1) { tmp = x1; x1 = x0; x0 = tmp; }
        if (y0>y1) { tmp = y1; y1 = y0; y0 = tmp; }
        for (x=x0; x<=x1; x++) {
                for (y=y0; y<=y1; y++)          {
                                draw_Pixel(x,y, fgColor, bgColor);
                                }
                }
}
```

draw vertical & horizontal lines

draw all pixels in the rectangle

main.c


```
int32_t main (void)
{
        UNLOCKREG();
        DrvSYS_Open(50000000);
        LOCKREG();

        init_LCD();
        clear_LCD();

        RectangleDraw(0,0,127,63, FG_COLOR, BG_COLOR); // draw a rectangle
        RectangleFill(10,10,20,20,FG_COLOR, BG_COLOR); // draw a box
        CircleBresenham(20,30,20,FG_COLOR, BG_COLOR);  // draw a circle
        CircleMidpoint(63,30,20,FG_COLOR, BG_COLOR);         // draw a circle
        CircleOptimized(105,30,20,FG_COLOR, BG_COLOR); // draw a circle
        LineBresenham(20,0,60,50,FG_COLOR, BG_COLOR);  // draw a line
        LineOptimized(30,20,100,40,FG_COLOR, BG_COLOR);// draw a line
        Triangle(20,40,90,10,60,60,FG_COLOR, BG_COLOR);// draw a triangle
}
```

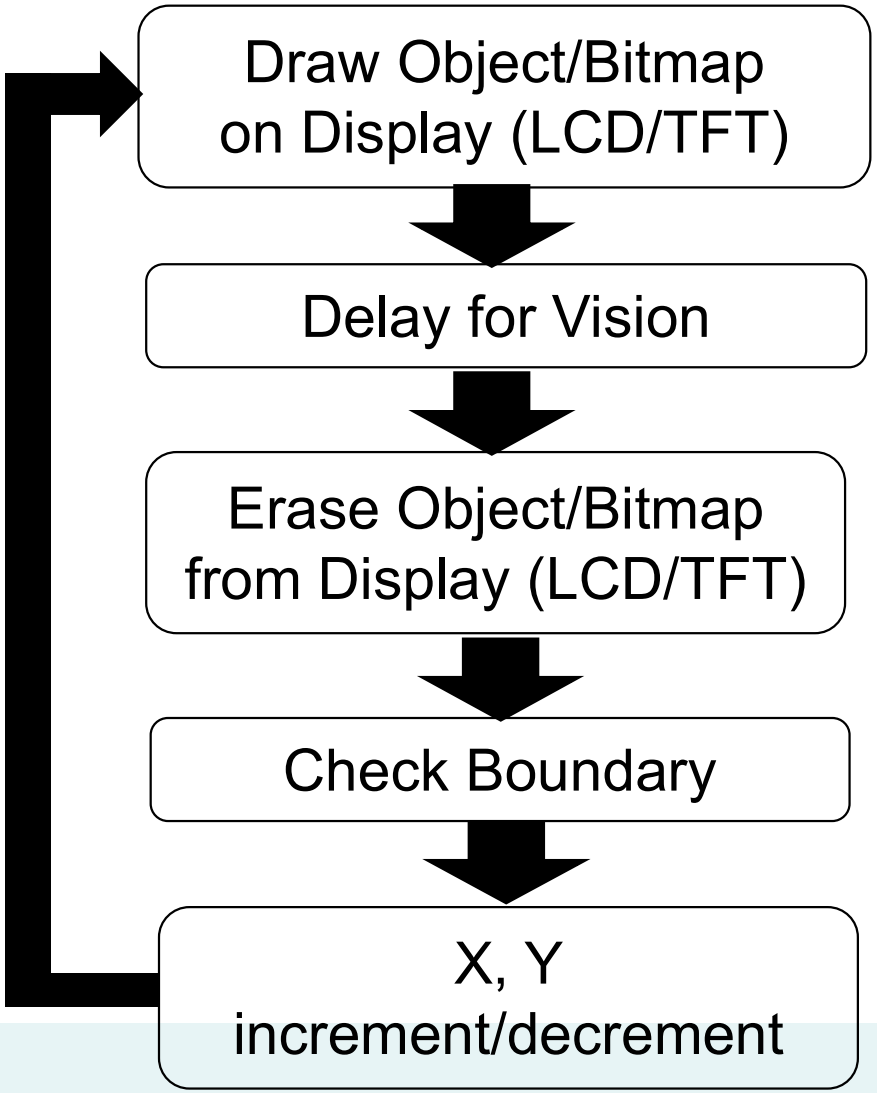
setting CPU clock

Initialize & clear LCD

Using 2D graphics driver to draw circle/line

Animation flow chart

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



main.c of Smp_LCD_Pingpong

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
int32_t main (void){
        int dirX, dirY;
        int movX, movY;
        uint16_t r;
        uint16_t x, y;
        uint16_t fgColor, bgColor;

        UNLOCKREG();
        DrvSYS_Open(50000000);
        LOCKREG();

        init_LCD();
        clear_LCD();

        x = X0;    // circle center x
        y = Y0;    // circle center y
        r = 5;    // circle radius
        movX = 3;  // x movement
        movY = 3;  // y movement
        dirX = 1;  // x direction
        dirY = 1;  // y direction

        bgColor = BG_COLOR;
```

set CPU running at 50MHz

initialize & clear LCD

initial value of x,y

initial value of x,y movement

initial value of x,y moving direction

initial background color (0x0000 for black)

Smp_LCD_Pingpong

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

```
while(1) {
        fgColor = FG_COLOR;
        CircleBresenham(x, y, r, fgColor, bgColor); // draw a circle
        LineBresenham(L_BAR_X,y-7, L_BAR_X,y+7, fgColor, bgColor); // draw left  line
        LineBresenham(R_BAR_X,y-7, R_BAR_X,y+7, fgColor, bgColor); // draw right line

        DrvSYS_Delay(100000); // adjustable delay for vision

        fgColor = BG_COLOR;
        CircleBresenham(x, y, r, fgColor, bgColor); // erase a circle
        LineBresenham(L_BAR_X,y-7,L_BAR_X,y+7, fgColor, bgColor); // draw left  line
        LineBresenham(R_BAR_X,y-7,R_BAR_X,y+7, fgColor, bgColor); // draw right line

        // boundary check for changing direction
         if      ((x-r) < L_BAR_X)  dirX=1;
           else if ((x+r) > R_BAR_X)  dirX=-1;
           else if ((y-r) < 0)        dirY=1;
           else if ((y+r) > LCD_Ymax) dirY=-1;

           x = x + dirX * movX; // change x of circle center
           y = y + dirY * movY; // change y of circle center
  }}
```

Draw Object

Delay for Vision

Erase Object

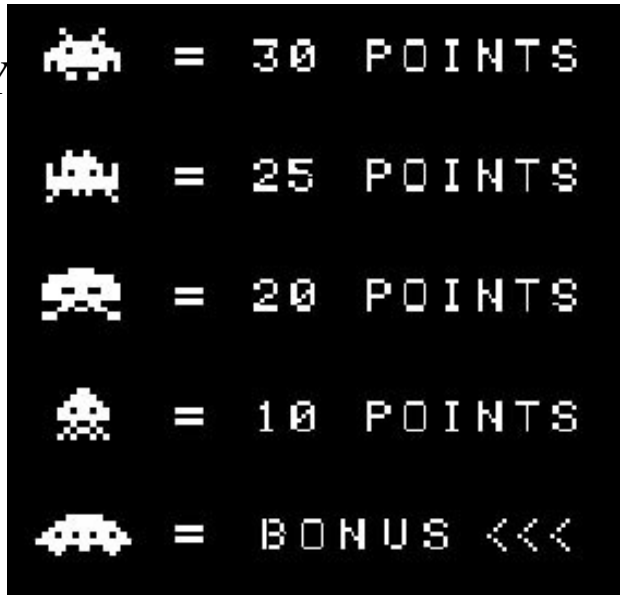
Check Boundary

Increment X,Y

Game Design Consideration

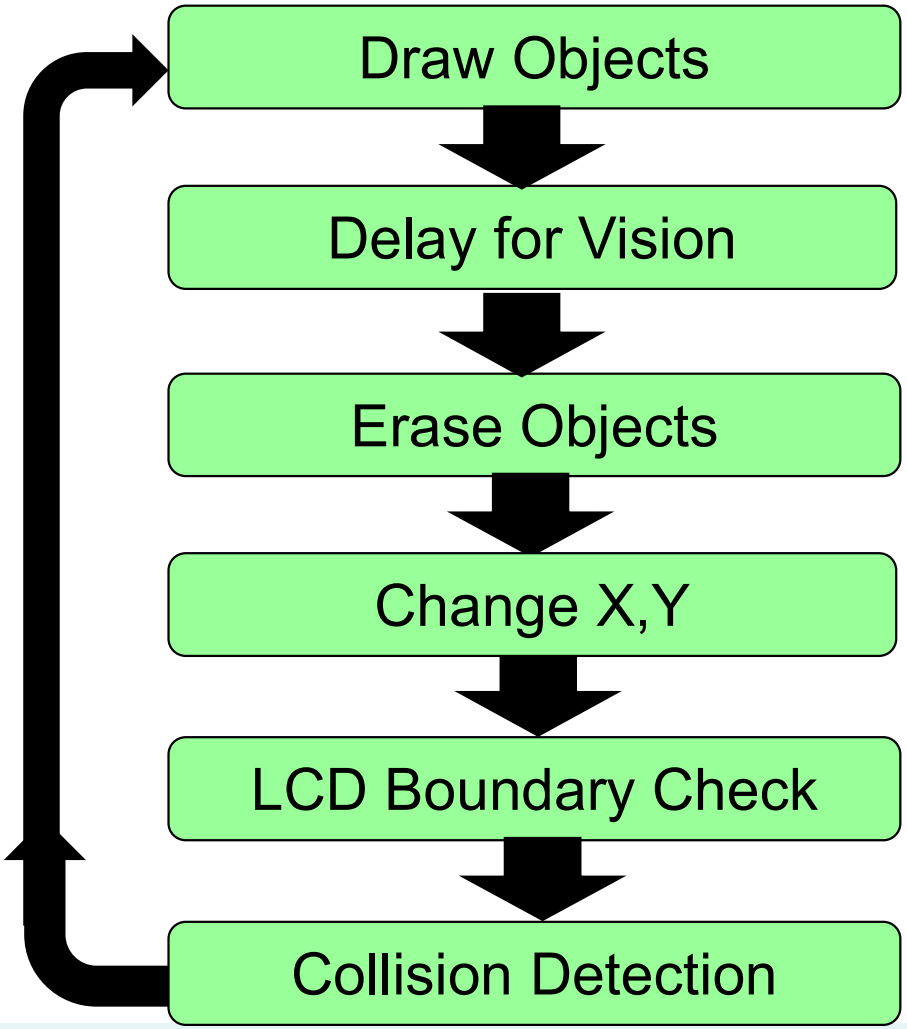
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- **Scene Size**
  - LCD 128x64 pixels : horizontal step X=128, vertical step Y
  - char display_buffer[128*8];
- **Create Object Bitmap**
  - Space Invader = **12x8 pixels**
- **Display Layout :**
  - Line Y=7 : Score & Life
  - Line Y=6 : UFO
  - Line Y=5~2 : Invaders
  - Line Y=0 : Cannon
- **Object movement**
  - 3x3 keypad input for increment/decrement of X,Y coordinate
- **Collision Detection**
  - Direct : compare object one-by-one
  - Map : collision map (array) to present object display or disapear (hit or destroyed)
- **Score & Life**



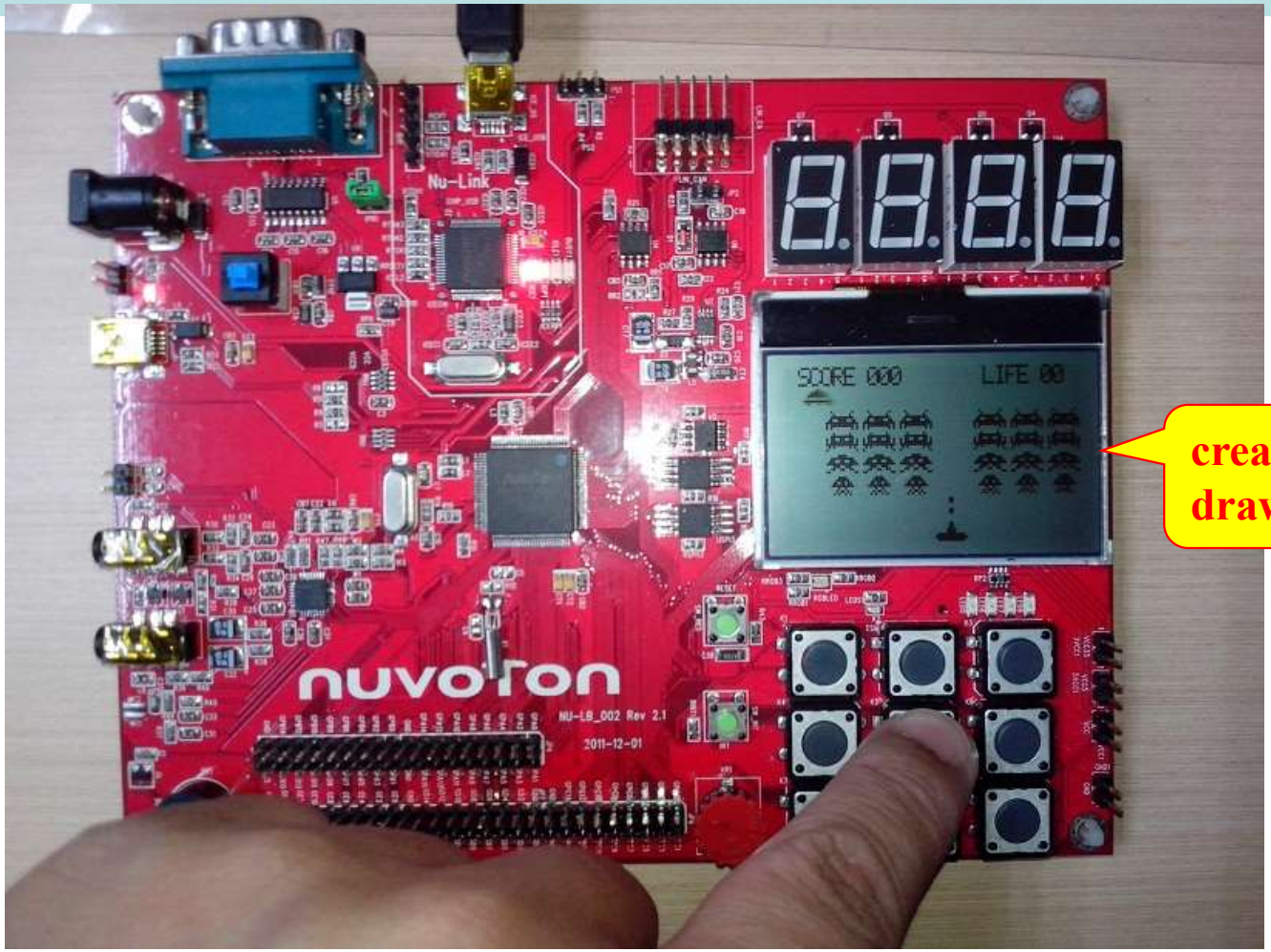
LCD Game flow chart

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



LCD Game – Space Invader

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

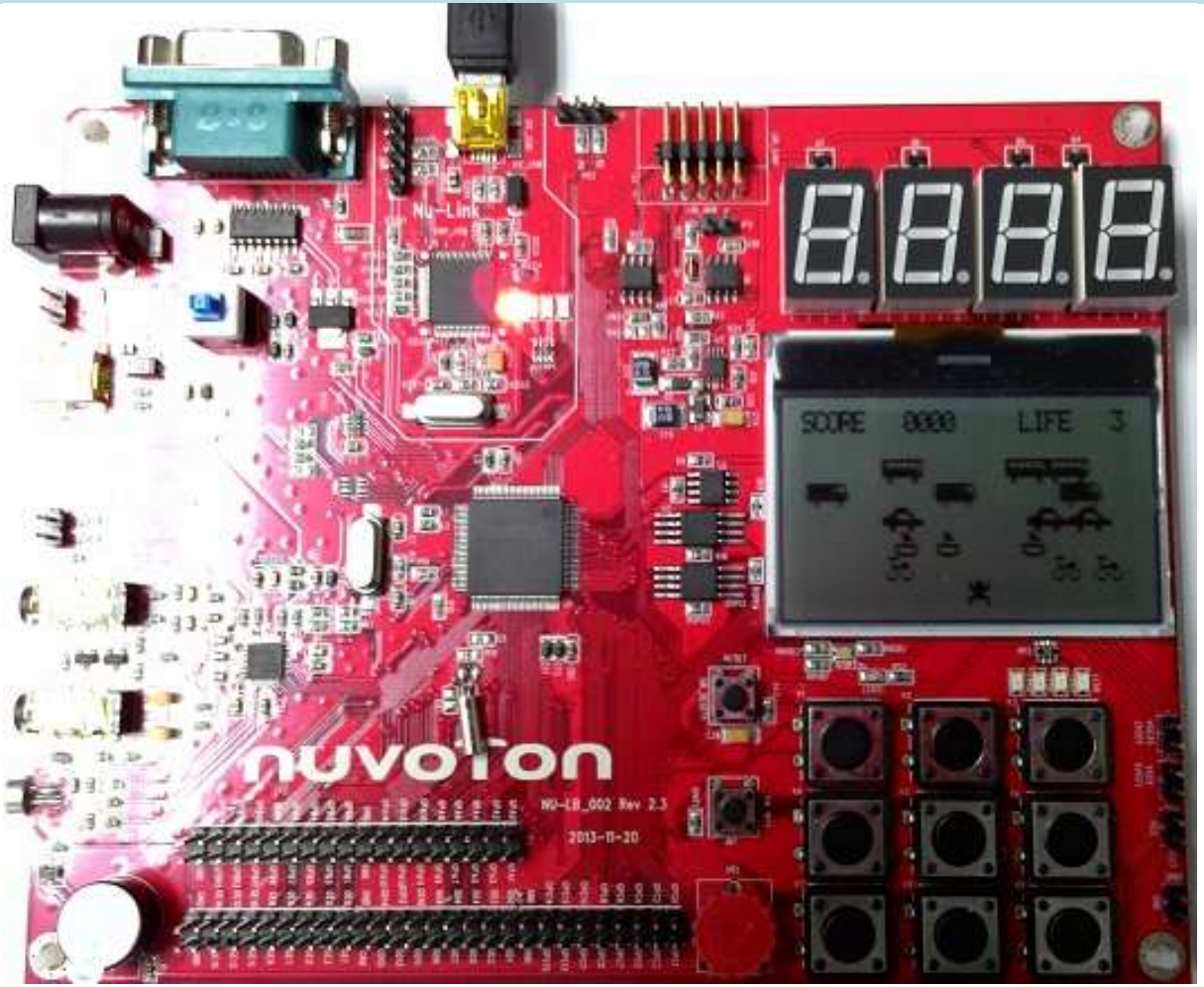


create Icon bitmaps
draw them onto LCD

D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduSample\Smpl_LCD_Game_Invader

LCD Game – Frogger

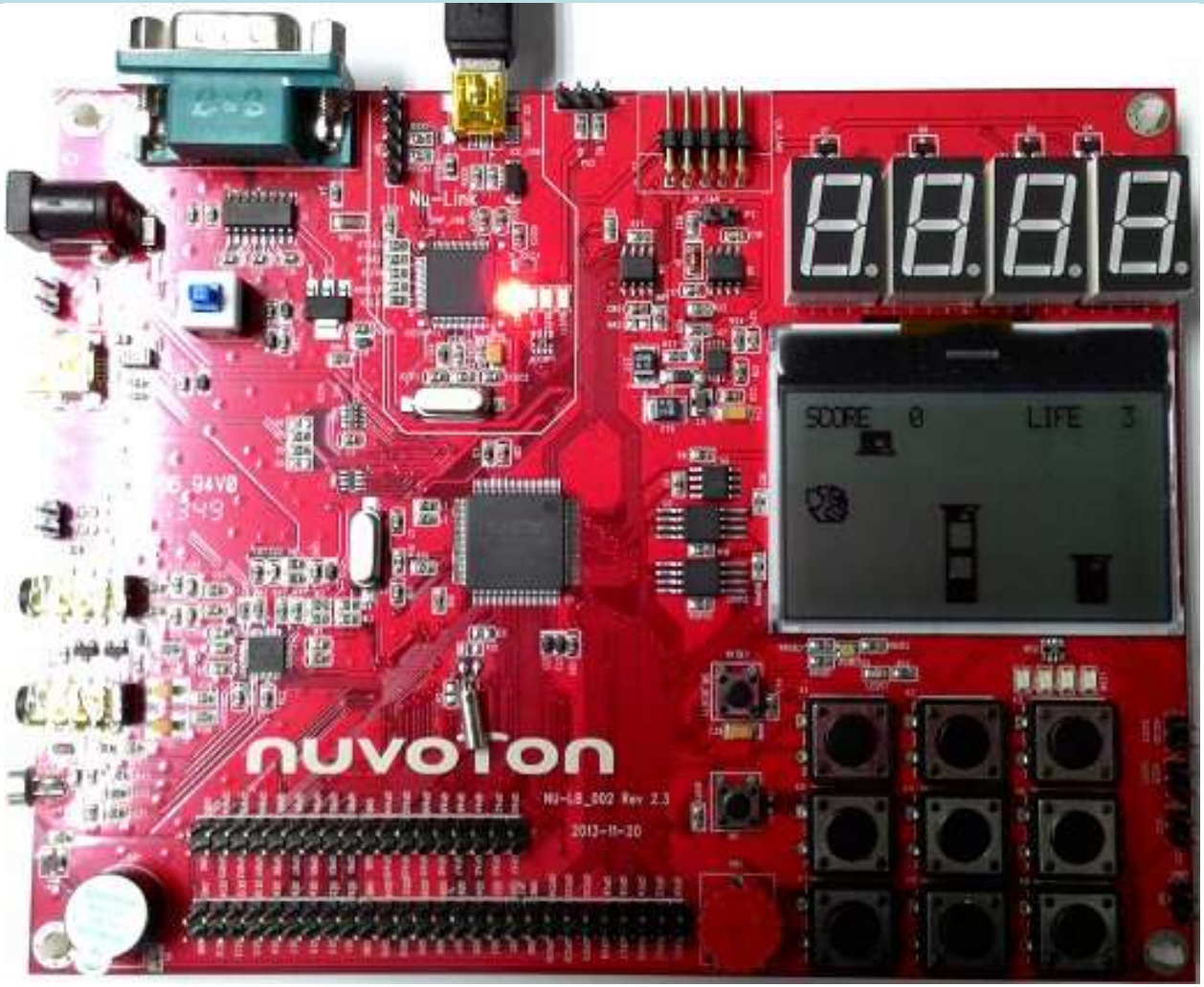
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduSample\Smpl_LCD_Game_Frogger

LCD Game – FlappyBird

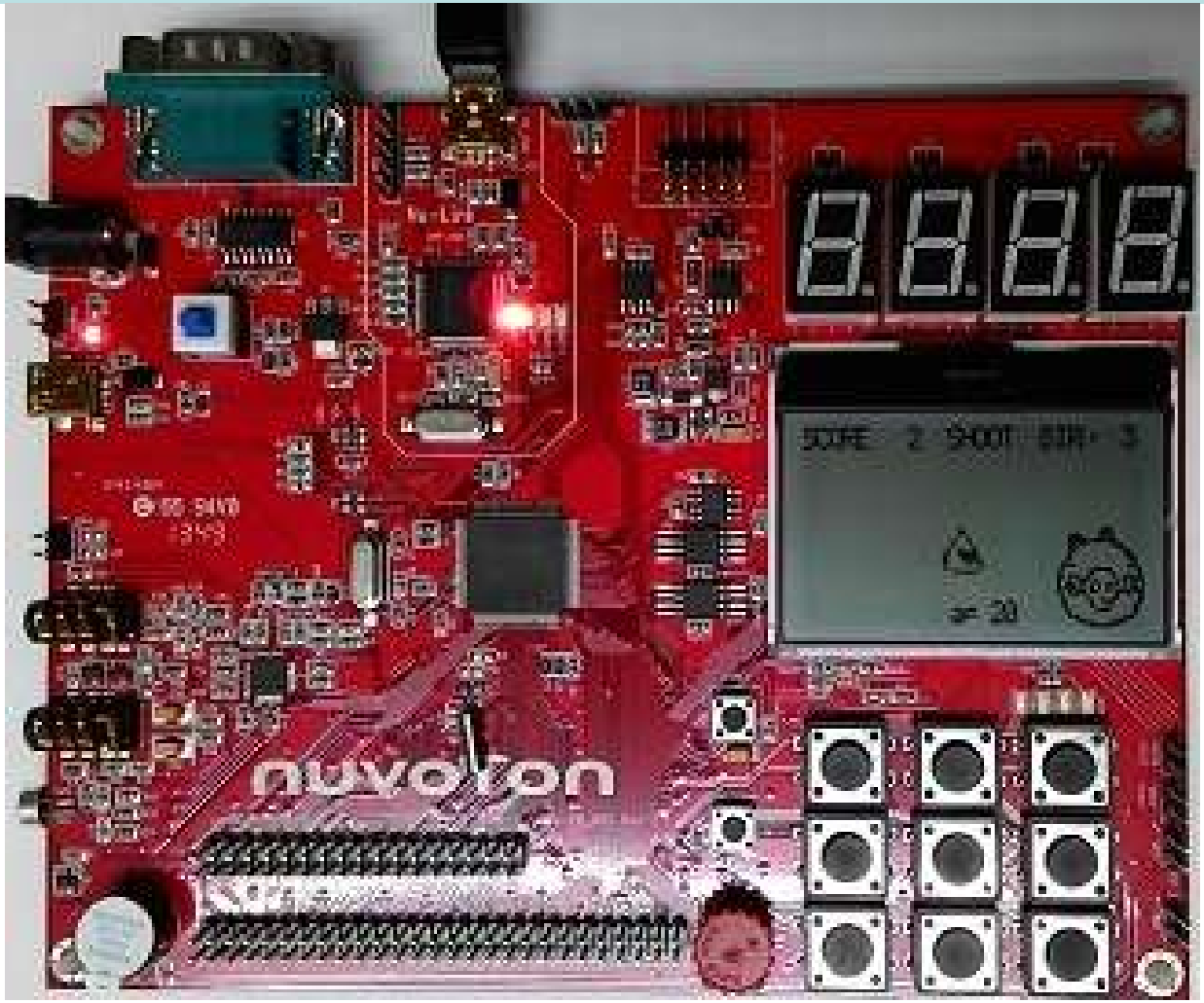
Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS



D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduSample\Smpl_LCD_Game_FlappyBird

LCD Game – AngryBird

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

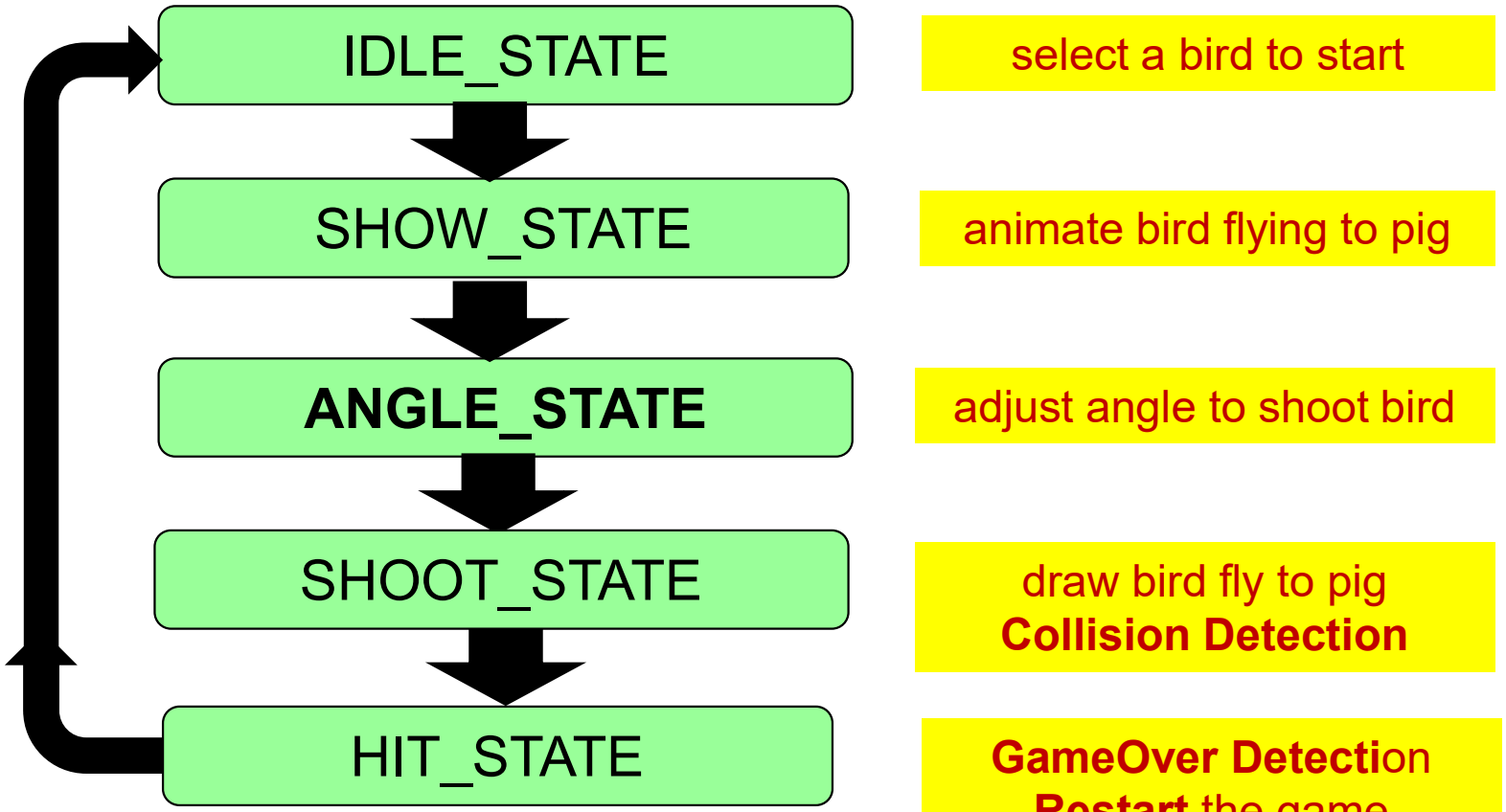


D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduSample\Smpl_LCD_Game_AngryBird

Game States

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS





Important Notice !

Department of Electrical and Electronic Engineering,
Universiti Teknologi PETRONAS

- **This educational material are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".**
- **Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.**
- **All Insecure Usage shall be made at user's own risk, and in the event that third parties lay claims to the author as a result of customer's Insecure Usage, the user shall indemnify the damages and liabilities thus incurred by using this material.**
- **Please note that all lecture and sample codes are subject to change without notice. All the trademarks of products and companies mentioned in this material belong to their respective owners.**